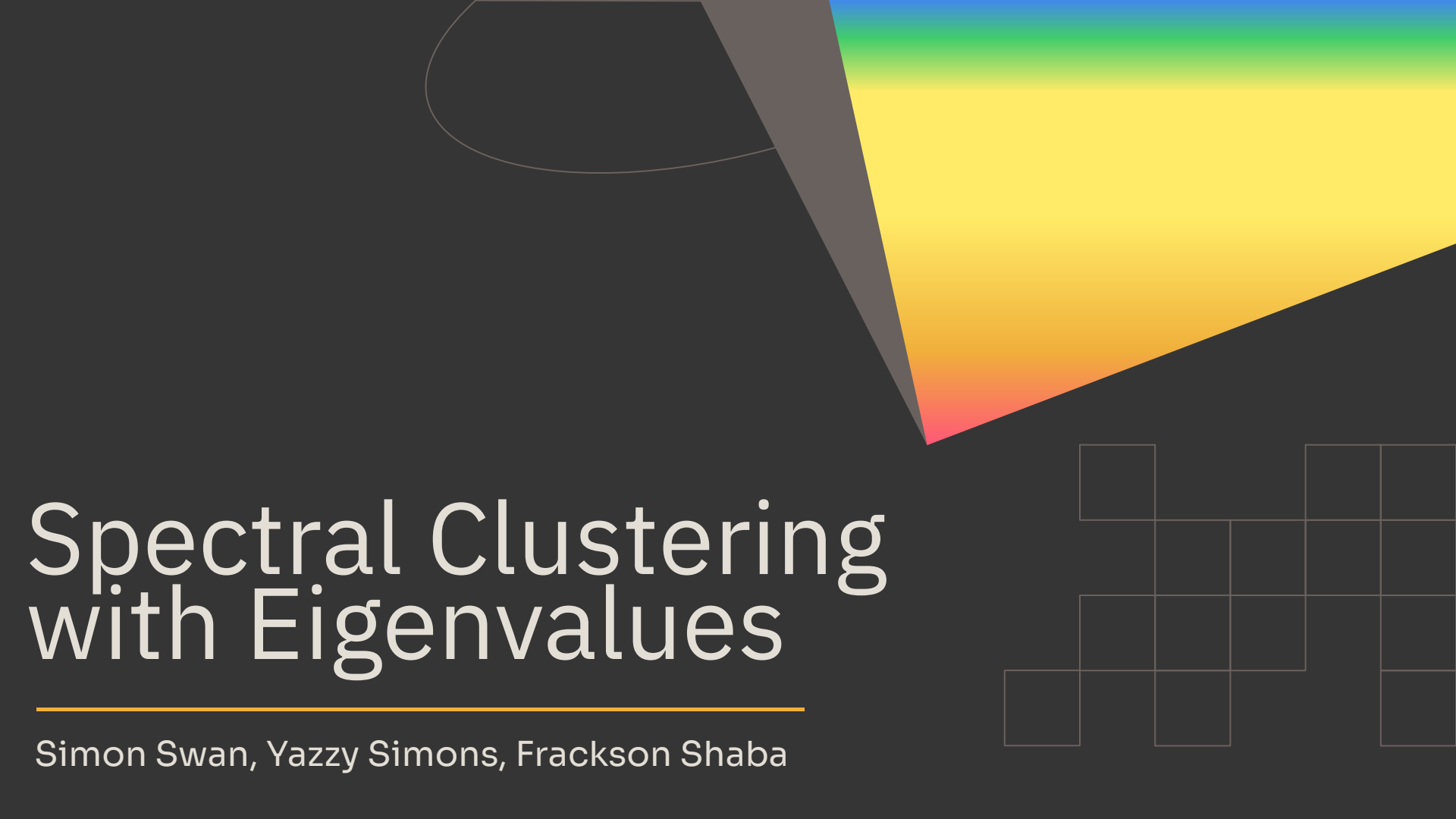


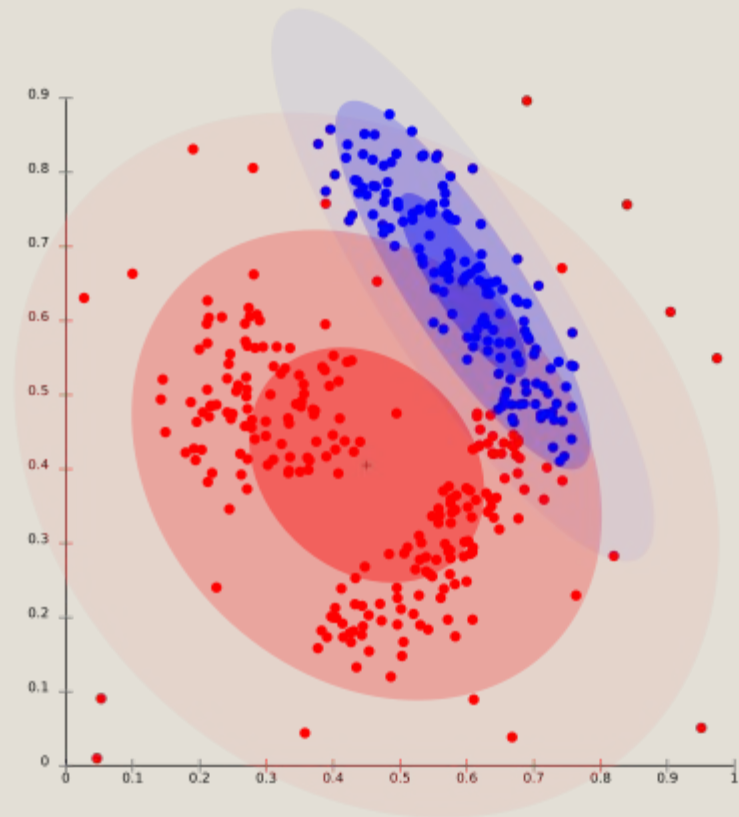


Spectral Clustering with Eigenvalues

Simon Swan, Yazzy Simons, Frackson Shaba

Clustering

(generally)



What is clustering?

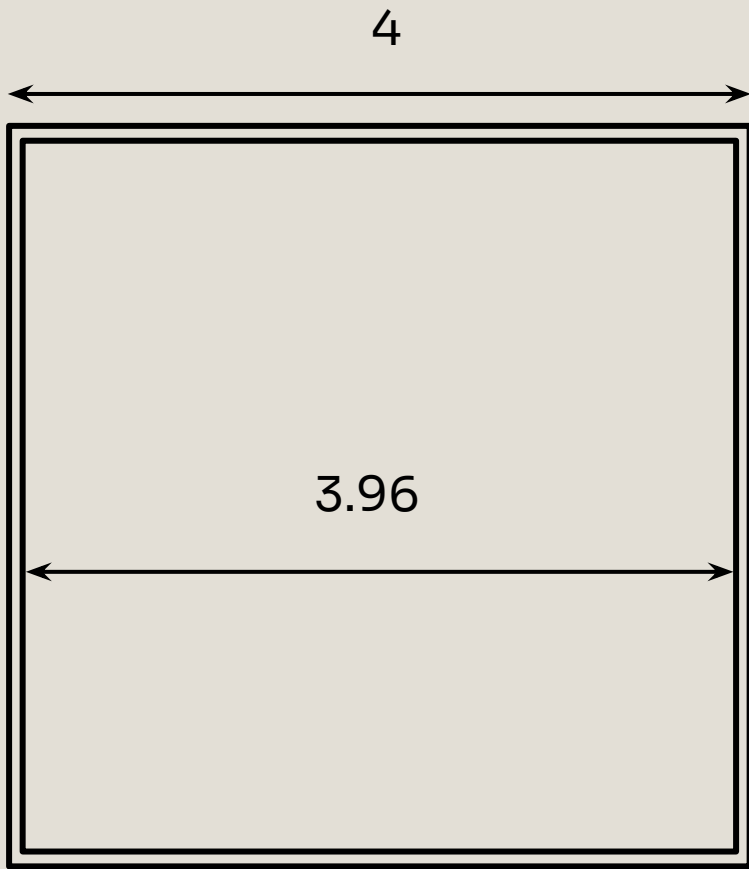
Overview

Grouping data so point in the same “cluster” are more similar to each other than points in other “clusters”

- ❑ Iterative and closed forms
- ❑ Unsupervised (ML context)
- ❑ Interpreting/visualizing data

Limitations

- ❑ Computationally expensive
- ❑ Curse of dimensionality
- ❑ Validation
- ❑ Invalid bias (issue with any ML task)
 - ❑ Choice of k
 - ❑ Expected cluster shape
 - ❑ Existence of meaningful clusters



$$\frac{3.96^2}{4^2} = (3.96/4)^2 = 0.9801$$

What happens in high dimension?

Let p be a point chosen uniformly at random from the outer square/cube/hypercube, let x be the length of one edge of the outer object, and y be the length of one edge of the inner object.

The probability that p falls within the inner object is $(y/x)^d$

$$\left(\frac{3.96}{4}\right)^{500} = 0.66\%$$

Why is clustering useful?

Benefits

- ❑ Does not require labeled data
- ❑ Reduces assumptions

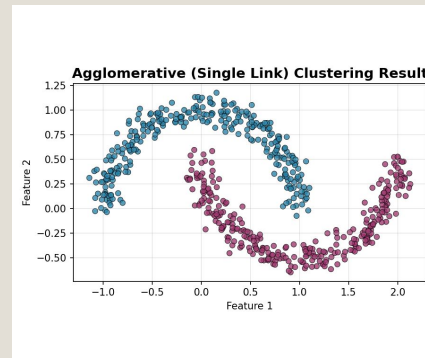
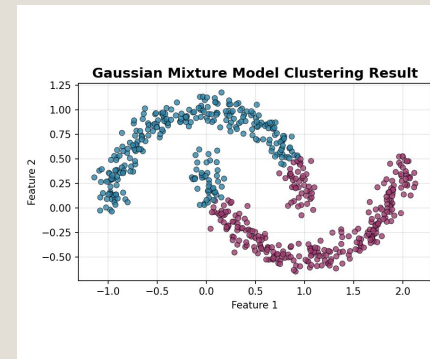
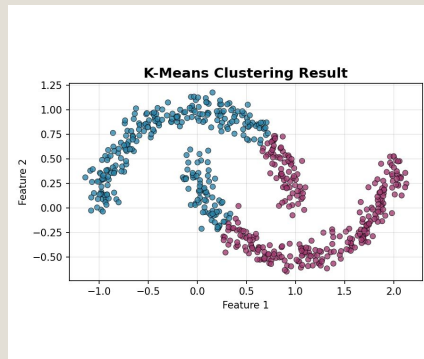
Example Use Cases

- ❑ Image compression
- ❑ Recommendation systems/advertising
- ❑ Classify unlabeled data

Alternatives

- ❑ Manual grouping (subjective, high bias)
- ❑ Supervised learning - classification (requires labels)
- ❑ Simple thresholds (low complexity, high bias)

Common Methods



K-means Clustering

Overview

- ❑ Minimize cost function C
- ❑ Iterative
- ❑ Training $\in O(tNdk)$
- ❑ Predict N points $\in O(Ndk)$
- ❑ Model complexity $\in O(dk)$
- ❑ Batch learning

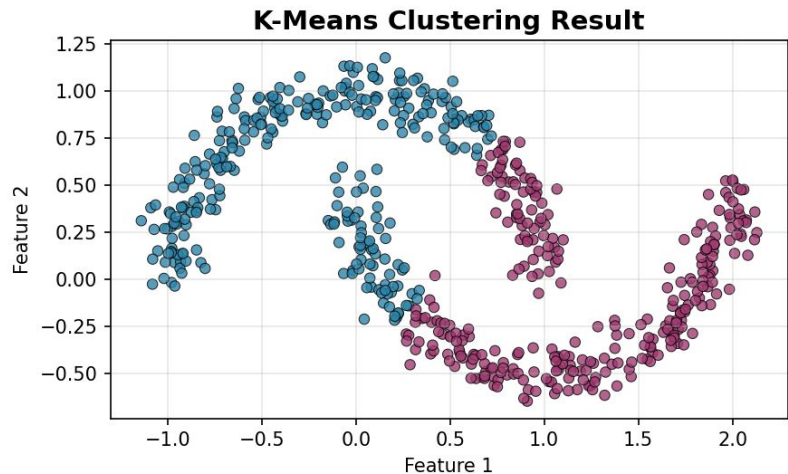
Benefit

- ❑ Known cluster centers

Drawback

- ❑ Breaks down for non-spherical clusters

$$C = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$



Gaussian Mixture Model Clustering

Overview

- ❑ Maximize $P(X|\theta)$
- ❑ Iterative
- ❑ Training $\in O(tNk(d^2))$
- ❑ Predict N points $\in O(Nk(d^2))$
- ❑ Model complexity $\in O(k(d^2))$
- ❑ Batch learning

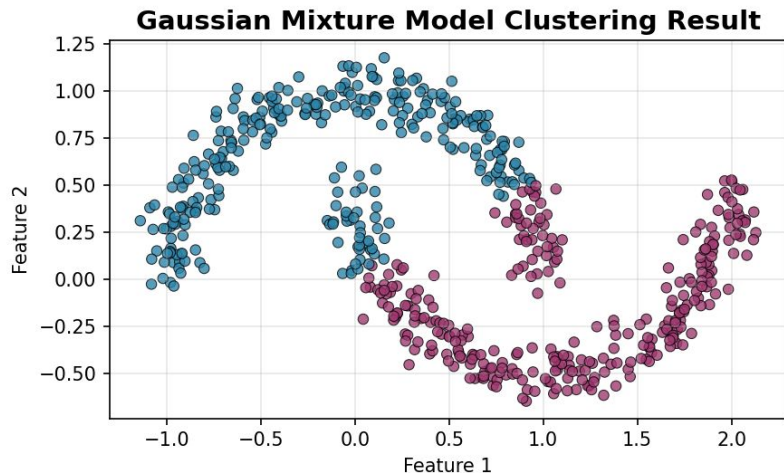
Benefit

- ❑ Soft cluster membership

Drawback

- ❑ Assumes clusters have normal distribution

$$P(X | \theta) = \prod_{i=1}^N \sum_{j=1}^k \pi_j \mathcal{N}(x_i | \mu_j, C_j)$$



Agglomerative Clustering

Overview

- ❑ Repeat merge step until $|C|=k$
- ❑ Incremental/Greedy
- ❑ Training (typical) $\in O(\log(n)(n^2))$
 - ❑ Worst case $\in O(n^3)$
- ❑ No parameters, cannot predict
- ❑ Model complexity $\in O(n)$

Benefit

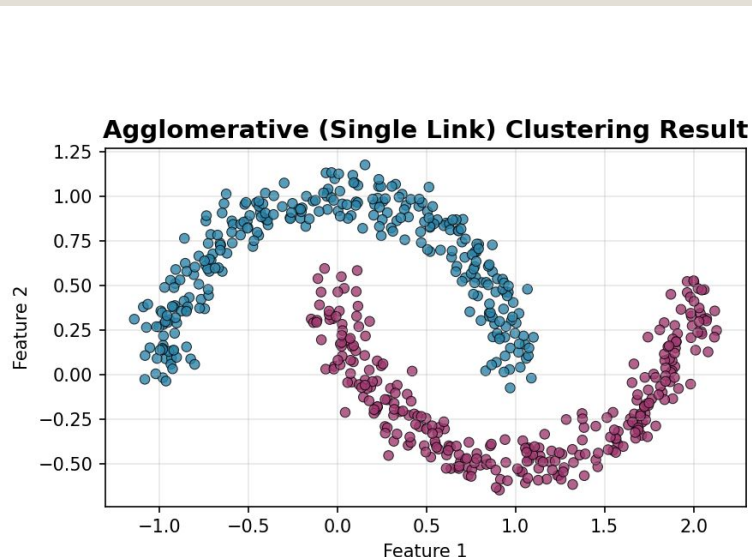
- ❑ Handles abnormal cluster shapes

Drawback

- ❑ Greedy - no backtracking/changing decisions

$$L_{single}(C_i, C_j) = \min_{x \in C_i, y \in C_j} \|y - x\|$$

$$Merge(C_i, C_j) | i, j = \underset{i, j}{\operatorname{argmin}} (L(C_i, C_j))$$



Spectral Clustering with Eigenvalues

Spectral Clustering

Overview

- ❑ Build W (k-NN + RBF)
- ❑ Form L_{sym} from W
- ❑ Take smallest eigenvectors U
- ❑ k-means on rows of U

Benefits

- ❑ Separates non-convex shapes
- ❑ Uses local geometry
- ❑ Simple k-means

Drawbacks

- ❑ Set k, σ for W
- ❑ Eigen-solve cost on large n
- ❑ Quality depends on W

$$L_{\text{sym}} = D^{-\frac{1}{2}} (D - W) D^{-\frac{1}{2}}$$

```
n_neighbors = 10

A = neighbors_graph(
    X,
    n_neighbors=n_neighbors,
    mode="distance",
    include_self=False
)
A = 0.5 * (A + A.T)

d = A.data
sigma = float(np.median(d)) if d.size else 1.0
W_data = np.exp(-(d**2) / (2 * sigma**2))

W = csr_matrix((W_data, A.indices, A.indptr), shape=A.shape)
W = 0.5 * (W + W.T)

save_npz("graphs/W_moons.npz", W)
with open("graphs/params.json", "w") as f:
    json.dump([
        {"moons": {"n_neighbors": n_neighbors, "sigma": "median_neighbor_distance"}},
        f,
    ], f,
    indent=2
)
```

You, 3 days ago • Yazy's Portion

Data → Graph

- ❑ Data: Two-Moons — $n=600$, noise =0.08, seed =0
- ❑ k-NN graph: $k=10$; symmetrize $A \leftarrow \frac{2}{1}(A + A^\top)$
- ❑ Weights (RBF): $w_{ij} = \exp\left(-\frac{d_{ij}^2}{2\sigma^2}\right)$
- ❑ Scale: $\sigma = \text{median}\{d_{ij} : j \in N_k(i)\}$
- ❑ Repro: fixed seeds; W and parameters logged

Laplacian → Eigenvectors → Labels

- ❑ Degrees: $D = \text{diag}(W1)$
- ❑ Laplacian: $L = D - W$
- ❑ Normalized: $L_{sym} = D^{-\frac{1}{2}} L D^{-\frac{1}{2}}$
- ❑ Eigenvectors: take k=2 smallest of $L_{sym} \rightarrow U$
- ❑ Row-normalize: $U_i \leftarrow \frac{U_i}{\|U_i\|_2}$
- ❑ Cluster: k-means on rows of:
 $U (k = 2, n_{init} = 10, seed = 0)$
- ❑ Output: spectral embedding U and final labels

```
W = load_npz("graphs/W_moons.npz")

deg = np.array(W.sum(axis=1)).ravel()
deg_safe = np.where(deg > 0, deg, 1.0)
D_inv_sqrt = diags(1.0 / np.sqrt(deg_safe))
L = diags(deg) - W
Lsym = D_inv_sqrt @ L @ D_inv_sqrt

sym_ok = (Lsym - Lsym.T).nnz == 0
print("L_sym symmetric:", sym_ok)

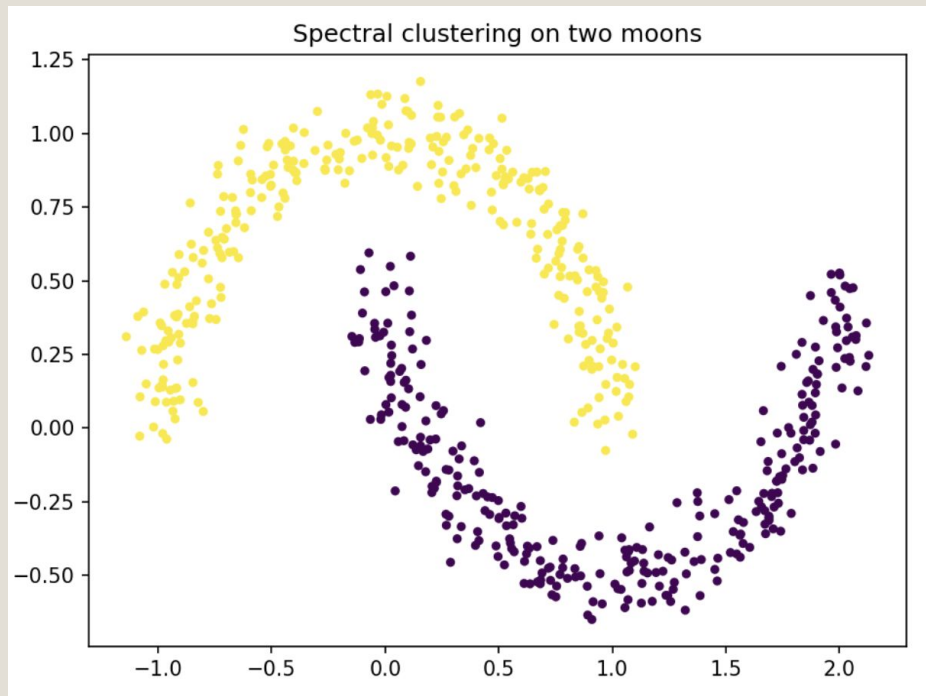
k = 2
eigvals, eigvecs = eigsh(Lsym, k=k, which="SM")
print("Eigenvalues (smallest k):", eigvals)

U = eigvecs / (np.linalg.norm(eigvecs, axis=1, keepdims=True) + 1e-12)

km = KMeans(n_clusters=k, n_init=10, random_state=0)
labels = km.fit_predict(U)
```

Result

- ❑ Spectral separates the two curved halves
- ❑ Params: $k=10$, σ = median neighbor distance, seed =0
- ❑ Why it matters: raw k-means on (x,y) often slices a moon



Clustering Methods Comparison Report

- There are differences between our clustering approach (Spectral Clustering) and
- clustering methods K-Means, Gaussian Mixture Model, and Agglomerative Clustering
- Two Moons dataset is ideal For testing if clustering methods can capture nonlinear structures
- Two curved clusters that overlap in the feature space

Differences

- K-Means assumes that clusters are circular or spherical in shape
- Separates data based on the distance from a central point (the centroid).
- It struggles with non-linear data like the two moons.
- In the visualization, K-Means splits the moons incorrectly along a straight line
- In our analysis resulting in a low Adjusted Rand Index ($AR = 0.25$)
- Poor normalized performance (0.19).
- Even though its silhouette score (0.48) looks decent
- That only shows that the clusters are compact internally not that they match the true pattern

Spectral Clustering works differently

- It first builds a similarity graph between points
- **Transforms it into a new space using eigenvectors**
- Then applies K-Means in that reduced space.
- This enables it to capture the curved structure of the moons.
- As a result, it correctly separates the two shapes
- Achieving an Adjusted Rand Index and Normalized Score of 1.0. Its silhouette score (0.33) is slightly lower because the edges of the moons are close together
- Overall, it provides more accurate clustering.

The numerical comparison confirms these differences:

- % of points clustered differently (Spectral vs K-Means): 24.83%
- % of points clustered differently (Spectral vs Gaussian Mixture): 15.17
- % of points clustered differently (Spectral vs Agglomerative): 0.00%

- In our project, the 24.83% difference between Spectral Clustering and K-Means
 - K-Means fails to capture non-linear relationships
- Spectral Clustering uses connectivity rather than raw distance.
- The smaller difference with the Gaussian Mixture Model (15.17%) shows that GMM captures some curvature
 - However not enough to fully model the complex shape.
- The 0.00% difference with Agglomerative Clustering suggests that both methods identified the same non-linear structure.

In summary

- In summary, the results show that Spectral Clustering is much better suited for datasets with curved or complex shapes
- K-Means and similar basic methods are limited to simpler, linearly separable data.
- The differences we observed in our analysis are consistent with these theoretical and practical distinctions

Thank you

